

DeployFlow: Blue-Green Deployment for ASP.NET Core on a Single VPS

For small teams running a .NET API on one server who release by hand over SSH and hope nothing breaks. A push to **main** becomes a tested, health-checked release with a one-command way back — no Kubernetes, no extra services.



THE PROBLEM

- Releases are hand-typed SSH commands that live in one person's head
- The old container is stopped before the new one has proven it works, so a bad build becomes an outage
- Nothing links the running container to a commit, and there's no quick way back

HOW IT WORKS

Build once, deploy the artefact

GitHub Actions builds and tests the API, then pushes a Docker image tagged with the commit SHA to GHCR. Nothing is built on the server.

New version proves itself first

The server starts the new image beside the live one, on a loopback-only port, and polls `/health`. Only a `200 OK` lets the release continue.

Controlled traffic switch

Nginx config is regenerated, validated with `nginx -t` and gracefully reloaded. The old container drains, then stops but is kept for rollback.

Failure and rollback

A failed health check removes the candidate and fails the pipeline — the live site keeps serving. `rollback.sh` restarts the previous release, re-checks it and switches back.

TYPICAL MANUAL SETUP VS. DEPLOYFLOW

TYPICAL MANUAL SETUP

- Hand-typed SSH commands
- Images built on the production server
- No answer to “which commit is live?”
- A bad build becomes a live outage
- Recovery means rebuilding under pressure

WITH DEPLOYFLOW

- A push to `main` is the whole procedure
- Immutable, SHA-tagged images from GHCR
- `/version` shows the live commit
- A bad build fails the pipeline, not the site
- One command restores the previous release

STACK

ASP.NET Core 8 · Docker (multi-stage, non-root, Alpine) · GitHub Actions · GHCR · Nginx with Let's Encrypt TLS · Bash · Ubuntu